CIS*3110 Operating Systems

Week 11

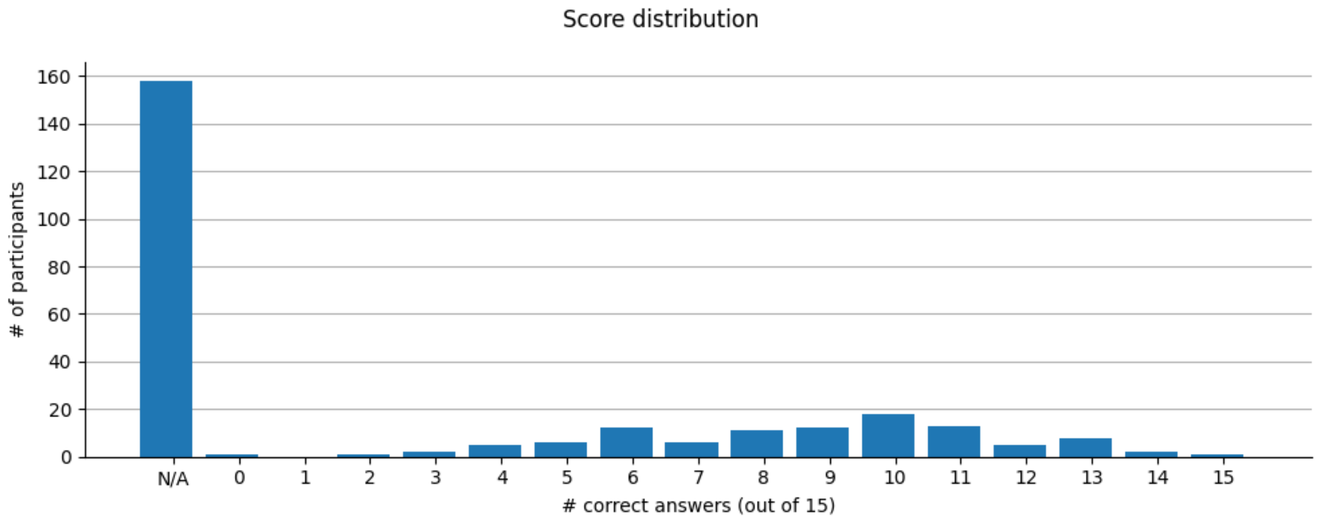
Lecture #17 Concurrency problems

Prof. Yulia Kotseruba

March 17, 2026

Quiz 15

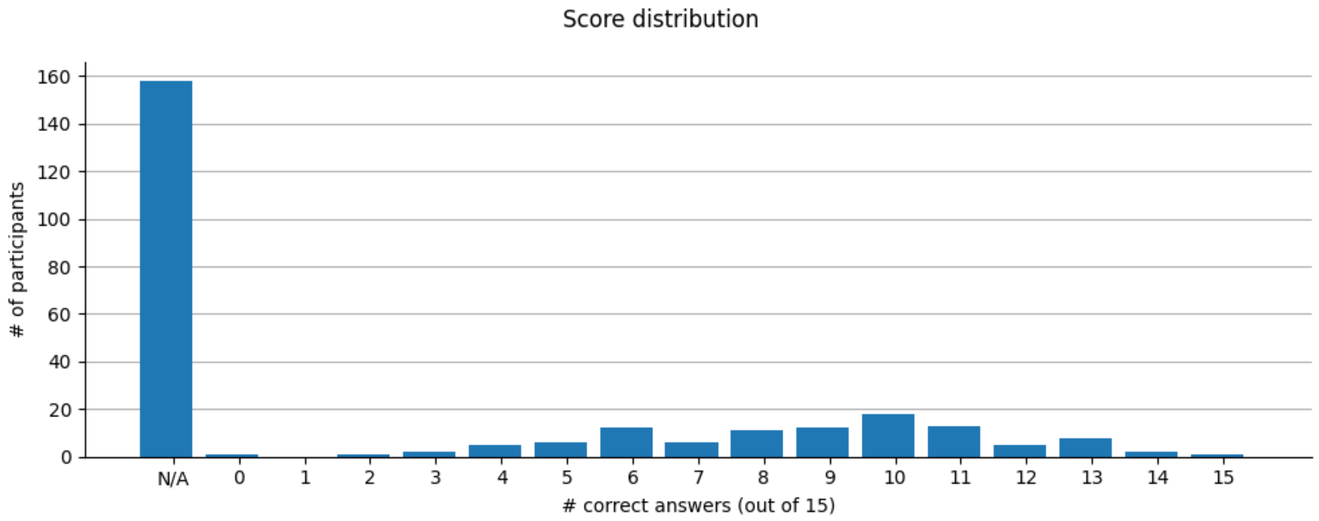
- **58%** average (down from 62% in quiz 13)
- 3 hardest questions:
  - (36%) How many times will A be printed? a) 1 b) 2 c) 3 d) any of the above
  - (36%) In Peterson's algorithm, when flag[0]=flag[1]=1, what determines which thread will enter CS? a) current value of turn b) who set the flag last c) CPU scheduler d) initial value of turn
  - (49%) Which process has a memory leak? a) child b) parent c) both d) neither




```c
void *inc(void *arg) {
    printf("A\n");
    exit(0);
}

int main(void) {
    int n = 3;
    pthread_t *t = malloc(n * sizeof *t);

    for (int i = 0; i < 3; ++i){
        pthread_create(&t[i], NULL, inc, NULL);
        pthread_join(t[i], NULL);
    }
    free(t);
    return 0;
}
```

Quiz 15

- **58%** average (down from 62% in quiz 13)
- 3 hardest questions:
  - (36%) How many times will A be printed? a) 1 b) 2 c) 3 d) any of the above
  - (36%) In Peterson's algorithm, when flag[0]=flag[1]=1, what determines which thread will enter CS? a) current value of turn b) who set the flag last c) CPU scheduler d) initial value of turn
  - (49%) Which process has a memory leak? a) child b) parent c) both d) neither

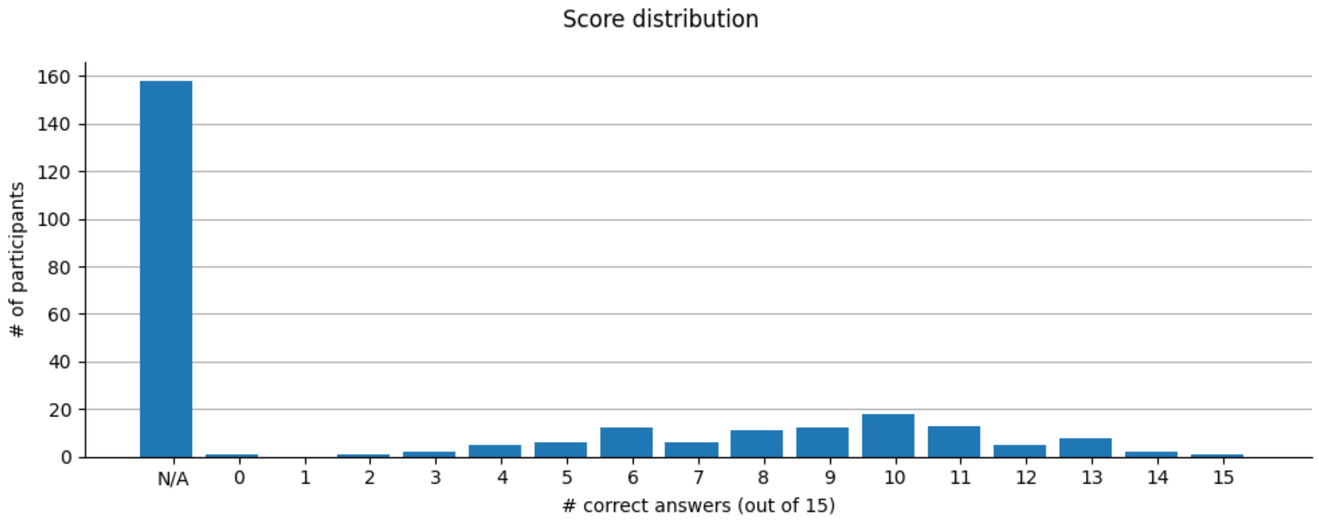



```c
void *inc(void *arg) {
    printf("A\n");
    exit(0);
}

int main(void) {
    int n = 3;
    pthread_t *t = malloc(n * sizeof *t);

    for (int i = 0; i < 3; ++i){
        pthread_create(&t[i], NULL, inc, NULL);
        pthread_join(t[i], NULL);
    }
    free(t);
    return 0;
}
```

exit(0) will kill the whole process with all its threads

Quiz 15

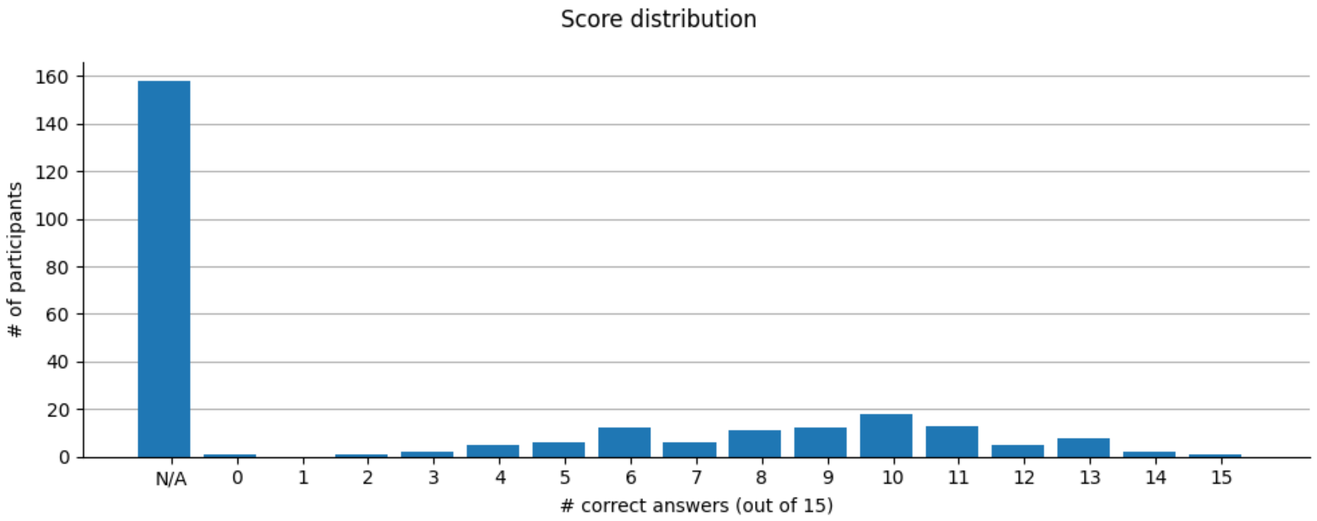
- **58%** average (down from 62% in quiz 13)
- 3 hardest questions:
  - (36%) How many times will A be printed? a) 1 b) 2 c) 3 d) any of the above
  - (36%) In Peterson's algorithm, when flag[0]=flag[1]=1, what determines which thread will enter CS? a) current value of turn b) who set the flag last c) CPU scheduler d) initial value of turn
  - (49%) Which process has a memory leak? a) child b) parent c) both d) neither




```
int main(void) {
    int *m = malloc(100);
    if (fork() > 0)
        execlp("echo", "echo", "B", NULL);
    free(m);
}
```

Quiz 15

- **58%** average (down from 62% in quiz 13)
- 3 hardest questions:
  - (36%) How many times will A be printed? a) 1 b) 2 c) 3 d) any of the above
  - (36%) In Peterson's algorithm, when flag[0]=flag[1]=1, what determines which thread will enter CS? a) current value of turn b) who set the flag last c) CPU scheduler d) initial value of turn
  - (49%) Which process has a memory leak? a) child b) parent c) both d) neither




```
int main(void) {
    int *m = malloc(100);
    if (fork() > 0)
        execlp("echo", "echo", "B", NULL);
    free(m);
}
```

parent will malloc and execlp, which will clean up everything -> no leak

child will malloc and free -> no leak

Recap

- Software-only locks do not work on modern hardware
- Atomic hardware instructions are the standard
  - Option #1 **test-and-set and spin** - not fair, not starvation-free, spinning wastes CPU
  - Option #2 **test-and-set and yield** - not fair, not starvation-free, less overhead
  - Option #3 **futex** - fair (uses a queue), starvation-free, efficient (one variable, few operations, minimizes kernel calls)
- Condition variables - useful for communication
  - require **both** locks and global state to work correctly

Recap

- **Producer/consumer** problem (aka bounded buffer problem)
  - multiple producer threads put values into the shared buffer
  - multiple consumer threads get values from the shared buffer
  - constraints: buffer is limited in size, producers cannot overwrite values in buffer, consumers cannot read what has not been produced

- **Semaphores**
  - single variable
  - sem_wait - decrements sem, waits if sem < 0
  - sem_post - increments sem, wakes up one waiting thread
  - can implement locks and condition variables
  - In current Linux kernel, POSIX semaphores are >= 0
    - this doesn’t change the logic

Today

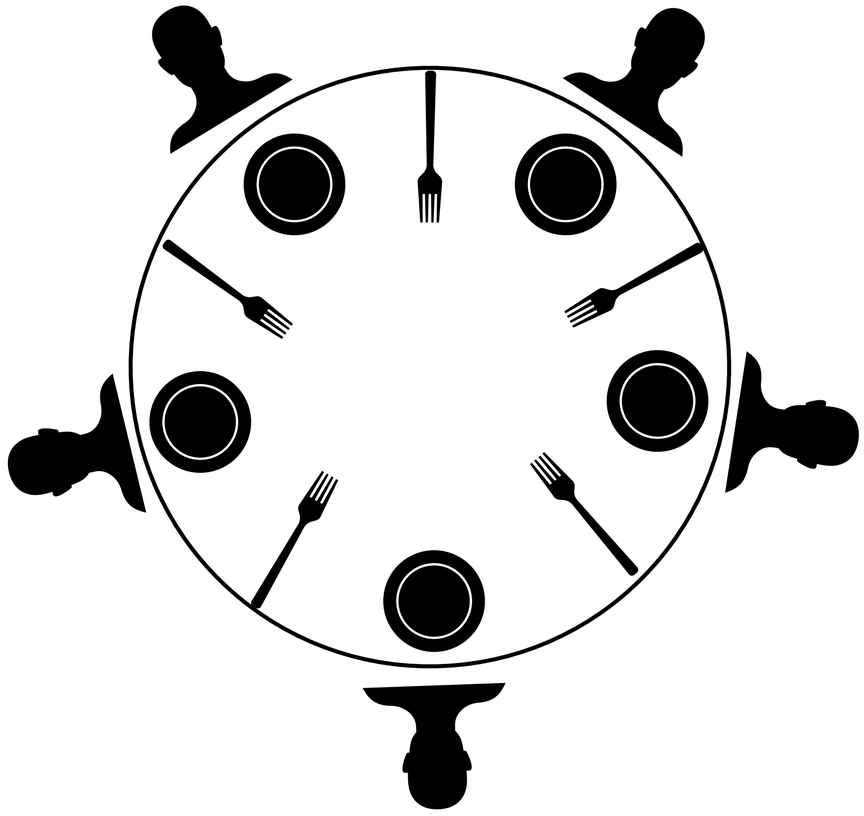
- More concurrency problems
  - Dining philosophers
  - Thread throttling
- Typical concurrency bugs and how to deal with them
  - Non-deadlock bugs
  - Deadlock bugs
- We are done with concurrency!

The Dining Philosophers

- The most famous concurrency problem
- Posed (and solved) by Dijkstra in 1965

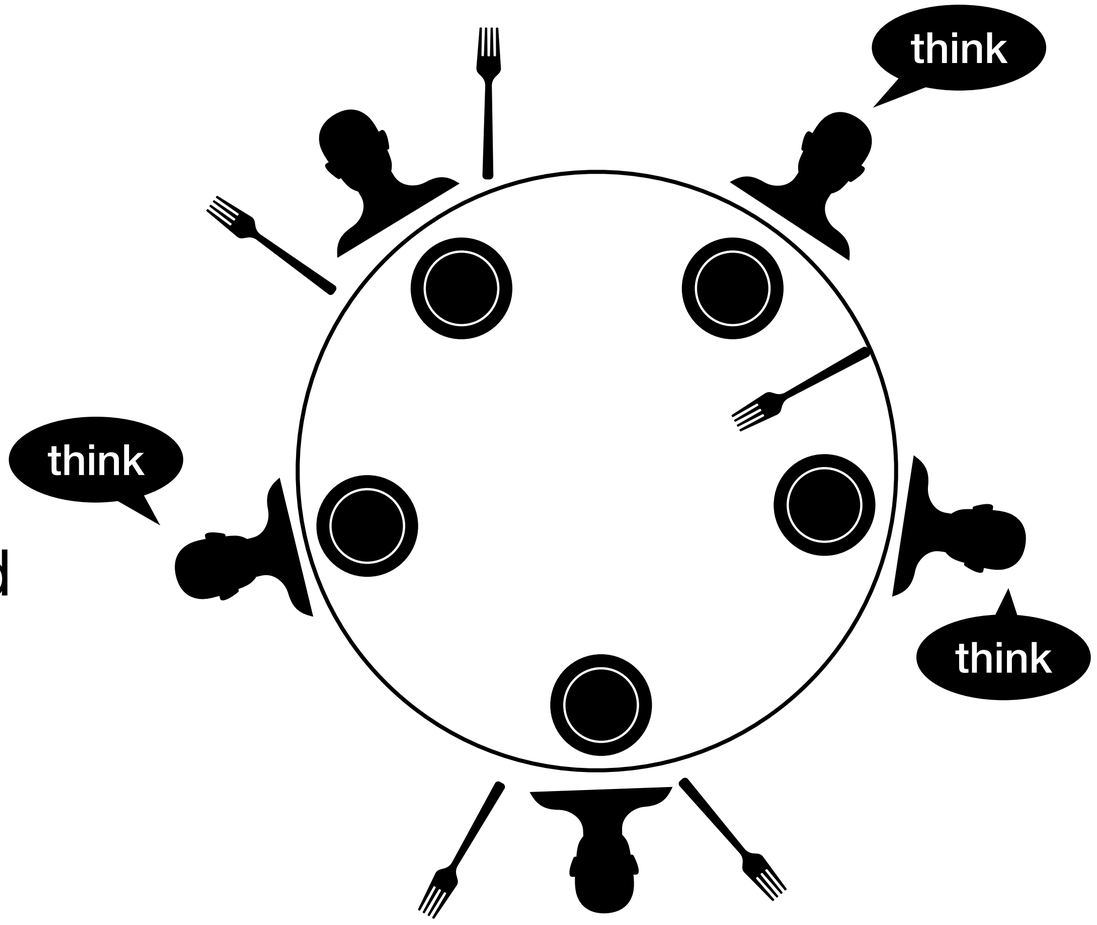
The Dining Philosophers

- Basic setup
  - 5 philosophers sitting around the table
  - 5 forks - one between each pair of philosophers
  - philosophers alternate between eating and thinking
    - thinking requires no forks
    - eating requires both forks



The Dining Philosophers

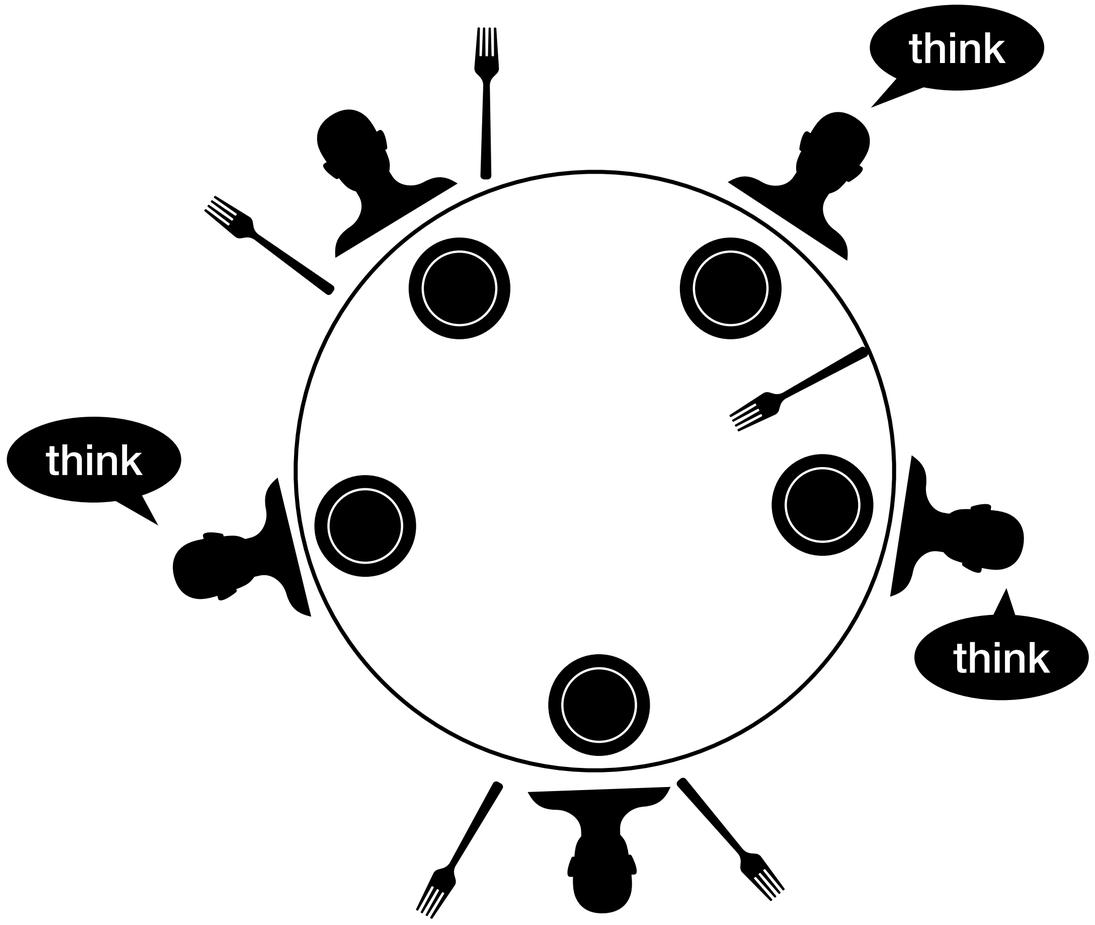
- Basic setup
  - 5 philosophers sitting around the table
  - 5 forks - one between each pair of philosophers
  - philosophers alternate between eating and thinking
    - thinking requires no forks
    - eating requires both forks



The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

- Implementation should
  - maximize concurrency -> feed as many philosophers as possible at the same time
  - no deadlocks -> ensure no philosopher starves



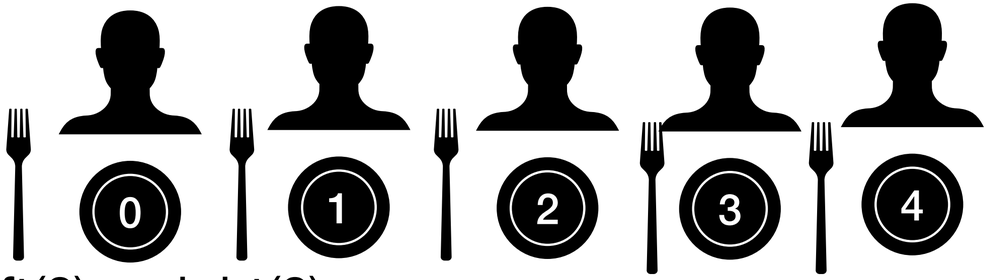
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }

int right(int p) {return (p+1) % 5;}
```

helper functions for philosophers to get forks to their right or left

easier to visualize as a circular array



left(0) right(0)

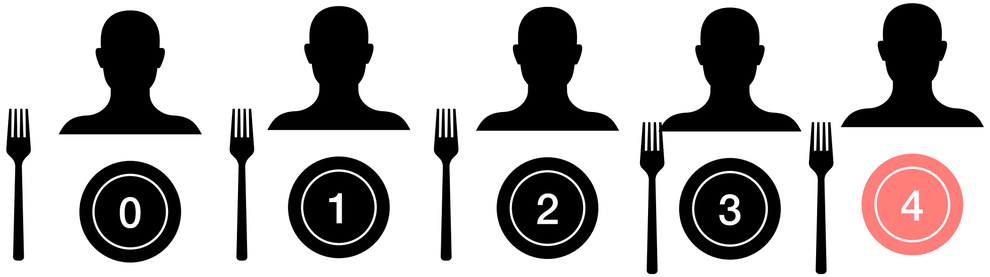
left fork of the Phil. #0

right fork of the Phil. #0

The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

Phil. #4 is a special case



right(4)
right fork of
the Phil. #4

left(4)
left fork of
the Phil. #4

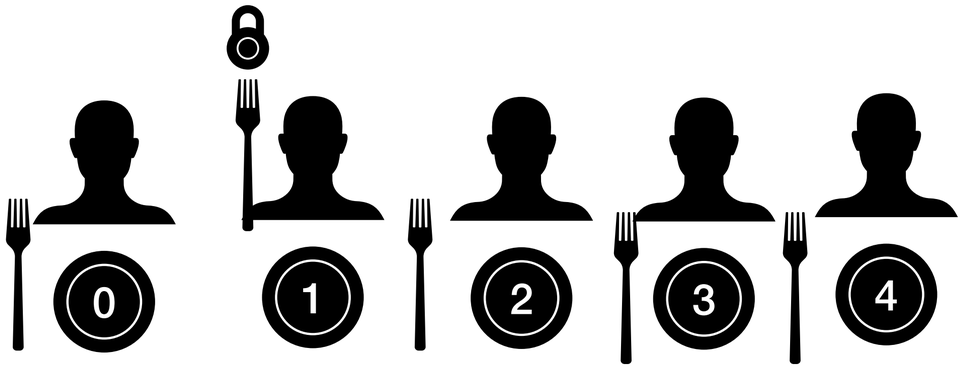
```
int left(int p) {return p; }

int right(int p) {return (p+1) % 5;}
```

needs a modulo to get the fork on the other side of the array

helper functions for philosophers to get forks to their right or left

The Dining Philosophers

when a philosopher grabs a fork to their left, it is locked, and others have to wait




```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

each philosopher will attempt to get left and right fork

use semaphores to lock the forks

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```

release both forks when done eating

this should work, right?

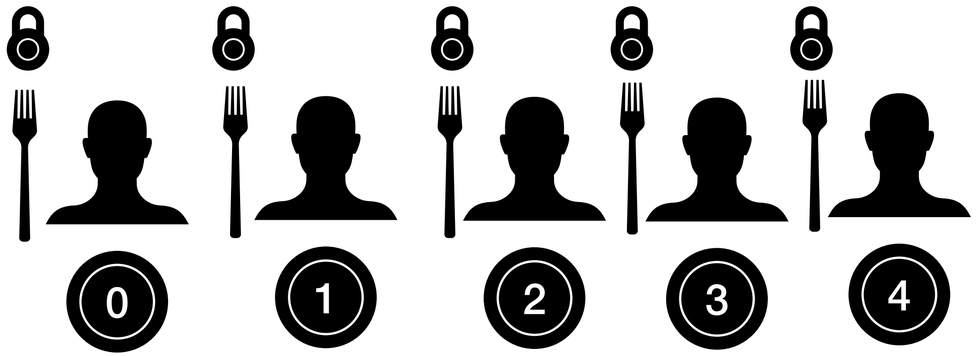
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



the solution breaks when all philosophers grab the left fork before they can grab the right fork

they are now deadlocked

how to fix?

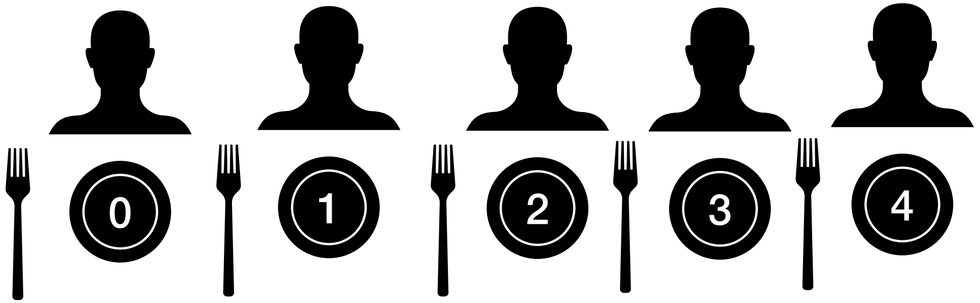
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



another possibility is livelock

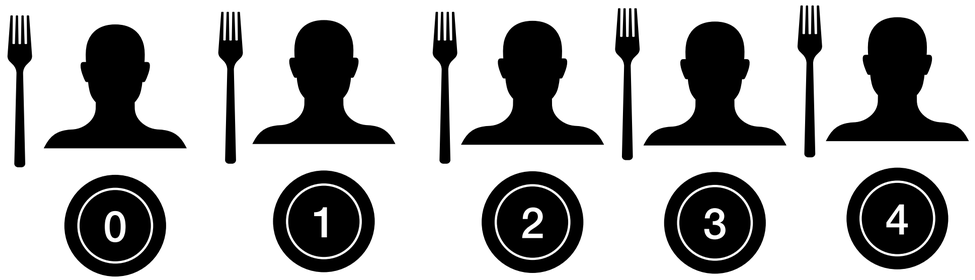
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



another possibility is livelock

if all philosophers grab a left fork

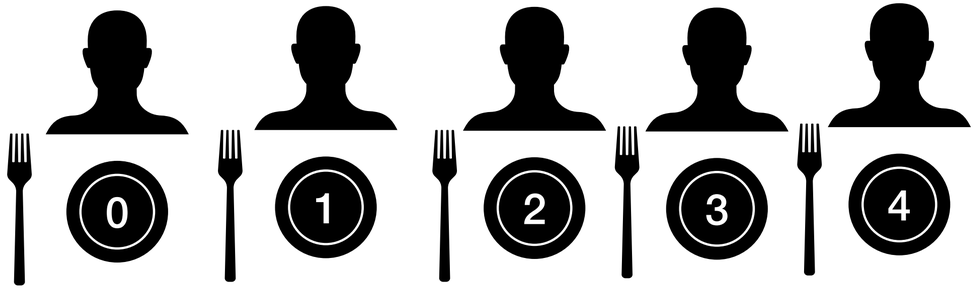
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



another possibility is livelock

if all philosophers grab a left fork then put it down

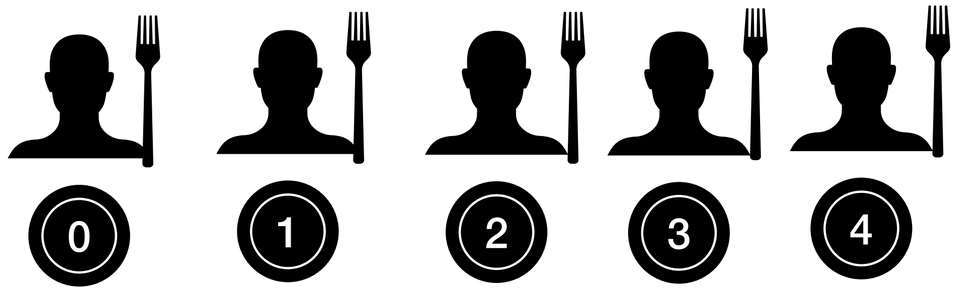
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



another possibility is livelock

if all philosophers grab a left fork
then put it down
and grab a right fork

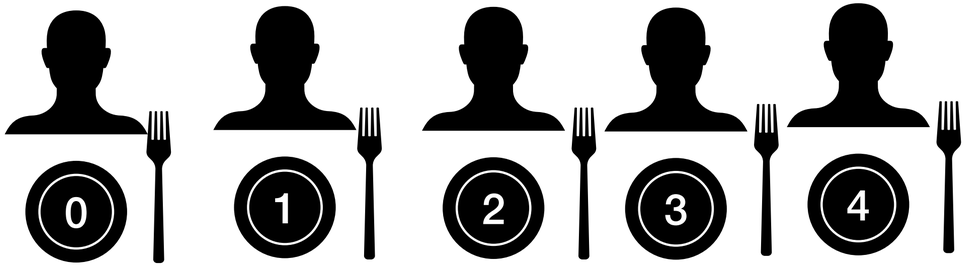
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



another possibility is livelock

if all philosophers grab a left fork
then put it down
and grab a right fork
then put it down
…

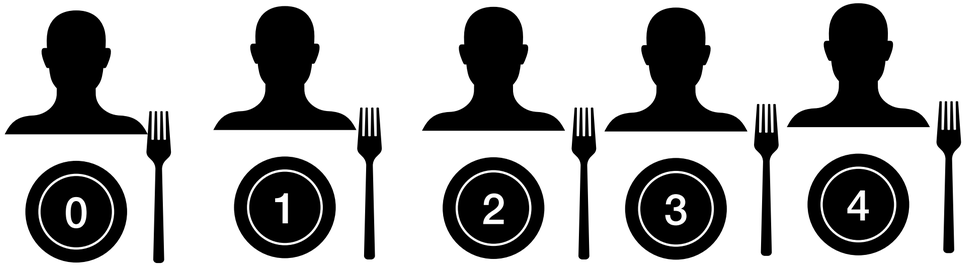
The Dining Philosophers

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
}
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  sem_wait(&forks[left[p]]);
  sem_wait(&forks[right[p]]);
}
```

```
void put_forks(int p) {
  sem_post(&forks[left[p]]);
  sem_post(&forks[right[p]]);
}
```



Deadlock - threads are stalled

Livelock - threads are running but are not making useful progress

The end result of both is the same -> the system is stuck

The Dining Philosophers

- The most famous concurrency problem
- Posed (and solved) by Dijkstra in 1965
- A clever illustration of deadlock/livelock, not practically significant
- Good to know for general education and interviews

Thread throttling

- Limit the number of threads that can access critical section
  - e.g. to protect memory-intensive regions of the code from exhausting RAM
    - which in turn can lead to thrashing

```
#define NUM_THREADS     5
#define MAX_CONCURRENT  2

static sem_t throttle_sem;


void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

there are 5 threads
but we want only 2 to run concurrently

the semaphore in worker limits the number of threads that can run inside

```
int main(void) {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];
    sem_init(&throttle_sem, 0, ) != 0);
    for (int i = 0; i < NUM_THREADS; ++i) {
         ids[i] = i + 1;
         pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; ++i) {
         pthread_join(threads[i], NULL);
    }
    sem_destroy(&throttle_sem);

    return 0;
}
```

main creates a bunch of threads and waits for them to join

```
#define NUM_THREADS     5
#define MAX_CONCURRENT  2

static sem_t throttle_sem;


void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

there are 5 threads
but we want only 2 to run concurrently

sem_wait decrements the value
sem_wait == 0 -> thread waits

sem_post increments the value
and wakes up one of the waiting threads

```
int main(void) {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];
    sem_init(&throttle_sem, 0,?) != 0);
    for (int i = 0; i < NUM_THREADS; ++i) {
         ids[i] = i + 1;
         pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; ++i) {
         pthread_join(threads[i], NULL);
    }
    sem_destroy(&throttle_sem);

    return 0;
}
```

what should we init the semaphore to for this to work?

main creates a bunch of threads and waits for them to join

```
#define NUM_THREADS     5
#define MAX_CONCURRENT  2

static sem_t throttle_sem;


void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

there are 5 threads
but we want only 2 to run concurrently

sem_wait decrements the value
sem_wait == 0 -> thread waits

sem_post increments the value
and wakes up one of the waiting threads

what should we init the semaphore to for this to work?

```
int main(void) {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];
    sem_init(&throttle_sem, 0, MAX_CONCURRENT) != 0);
    for (int i = 0; i < NUM_THREADS; ++i) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; ++i) {
        pthread_join(threads[i], NULL);
    }
    sem_destroy(&throttle_sem);

    return 0;
}
```

main creates a bunch of threads and waits for them to join

sem = 2

thread 1 starts

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

sem = 1

thread 1 starts
decrements sem
and continues

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
1   printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

sem = 1

thread 1 starts
decrements sem
and continues

thread 2 starts

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

2

1

sem = 0

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

2

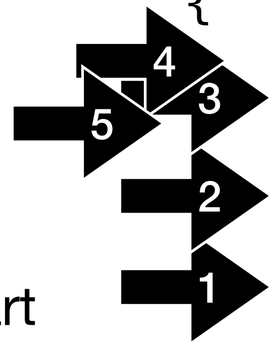
1

sem = 0

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

threads 3, 4, 5 start



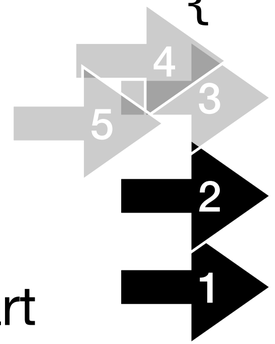

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

sem = 0

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

threads 3, 4, 5 start
sem == 0 -> have to
wait




```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

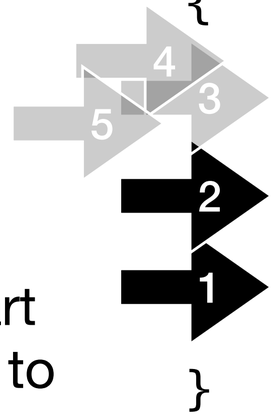
sem = 0

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

threads 3, 4, 5 start
sem == 0 -> have to
wait

thread 1 calls sem_post




```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

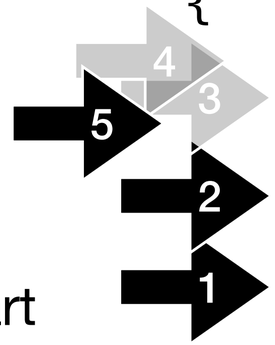
sem = 1

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

threads 3, 4, 5 start
sem == 0 -> have to
wait

thread 1 calls sem_post
increments sem and wakes up one of the waiting threads




```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

sem = 0

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

threads 3, 4, 5 start
sem == 0 -> have to
wait

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

thread 1 calls sem_post
increments sem and wakes up one of the waiting threads
thread 5 decrements sem and continues

sem = 0

setting semaphore to 2 ensures that no more than 2 threads will enter the critical section

thread 1 starts
decrements sem
and continues

thread 2 starts
decrements sem
and continues

threads 3, 4, 5 start
sem == 0 -> have to wait

```
void *worker(void *arg)
{
    int id = *(int *)arg;
    sem_wait(&throttle_sem);
    printf("Thread %d: started\n", id);
    sleep(1);
    printf("Thread %d: finished\n", id);
    sem_post(&throttle_sem);
    return NULL;
}
```

thread 1 calls sem_post
increments sem and wakes up one of the waiting threads
thread 5 decrements sem and continues

Thread throttling

- Limit the number of threads that can access critical section
- Can use a semaphore
  - to count how many threads are admitted
  - make other threads wait until at least one of the admitted threads exits

Summary so far

- Dining philosophers
  - maximize concurrent use of resources (forks) without starvation
- Thread throttling
  - control how many threads are working concurrently

Concurrency bugs

- Most **complex** applications use concurrency
  - e.g. browsers, database management systems, web servers, OSs, etc.
- Questions to ask about bugs
  - Q1: How **frequent** are concurrency bugs? Frequent enough to be a problem
  - Q2: What **types** of concurrency bugs are there?
  - Q3: Are concurrency bugs **harder** to fix? Likely yes, but difficult to quantify

Concurrency bugs

- Q2: What types of concurrency bugs are there?
  - **Non-deadlock**
    - Atomicity-violation - e.g. race conditions from incrementing shared variables
    - Order-violation bugs - e.g. signals getting lost or sent at a wrong time
  - **Deadlock**
- Non-deadlock bugs are **much more common** than deadlock bugs
- Deadlock bugs are rare but **much more severe**

Deadlocks

- Why do deadlocks occur?
  - Complex **dependencies** among large subsystems
  - **Encapsulation** - implementation details are hidden
    - a calling function is not always aware of the potential issues

Deadlocks

- Conditions for deadlocks
  - **Mutual exclusion** - threads claim exclusive control of resources they need
  - **Hold-and-wait** - threads hold resources allocated to them while waiting for additional resources
  - **No preemption** - resources cannot be forcibly removed from threads holding them
  - **Circular wait** - there is a circular chain of threads, where each thread holds resources needed by the next thread in the chain
- Deadlock occurs if **ALL** of these conditions are met

Example from Lecture #15

```
void *t1(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m1);
    usleep(100000);
    pthread_mutex_lock(&m2);
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}
```

t1 acquires m1 lock

takes a nap

t1 wakes up and starts waiting for m2 to be released

these two threads are now **deadlocked**

```
void *t2(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m2);
    pthread_mutex_lock(&m1);
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

second thread locks m2

and starts waiting for m1 to be released

```
pthread_mutex_t m1 =
PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 =
PTHREAD_MUTEX_INITIALIZER;
int main(void)
{
    pthread_t p1, p2;

    pthread_create(&p1, NULL, t1, NULL);
    pthread_create(&p2, NULL, t2, NULL);

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);

    return 0;
```

main creates two locks, two threads and waits for them to finish

```
void *t1(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m1);
    usleep(100000);
    pthread_mutex_lock(&m2);
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}
```

```
void *t2(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m2);
    pthread_mutex_lock(&m1);
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

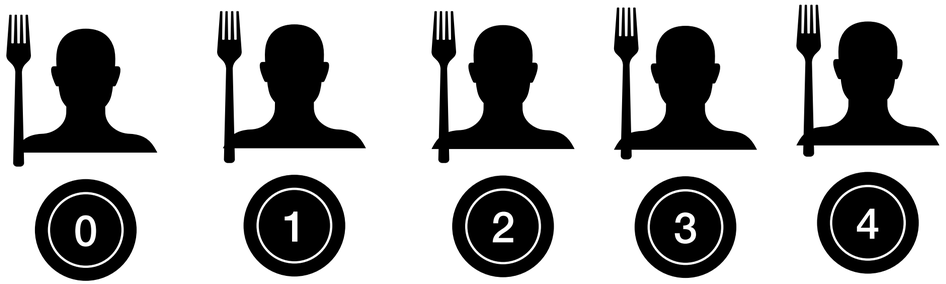
Example from Lecture #15

Deadlock conditions	
✔ mutual exclusion	both t1 and t2 need locks and hold them
✔ hold-and-wait	both t1 and t2 are need another lock and are waiting for it
✔ no preemption	nothing can release the resources to free t1 or t2
✔ circular wait	t1 waits on something that t2 is holding and vice versa

all conditions are satisfied -> deadlock is guaranteed

- **Mutual exclusion** - threads claim exclusive control of resources they need
- **Hold-and-wait** - threads hold resources allocated to them while waiting for additional resources
- **No preemption** - resources cannot be forcibly removed from threads holding them
- **Circular wait** - there is a circular chain of threads, where each thread holds resources needed by the next thread in the chain

Dining Philosophers
problem from earlier today



Deadlock conditions

✓ mutual exclusion	All philosophers need resources (forks) and are holding them
✓ hold-and-wait	All philosophers need another resource (a second fork) being held by someone else
✓ no preemption	No philosopher can give up their fork
✓ circular wait	Each philosopher waits for the next neighbour to give up their fork

Dealing with deadlocks

- Prevention - prevent one of the conditions from happening
- Avoidance - anticipate and work around deadlocks
- Let them happen and recover

Prevention: Circular wait

- Circular wait is the **easiest** to prevent
  - **Order** lock acquisition
    - If there are two locks L1 and L2, always acquire them in the same order
      - e.g. always L1->L2 or always L2->L1
    - Can be enforced by lock address (e.g. always grab in high-to-low address order)

Example from Lecture #15

```
void *t1(void *arg) {
    pthread_mutex_lock(&m1);
    usleep(100000);
    pthread_mutex_lock(&m2);
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}
```

How can we apply lock ordering here?

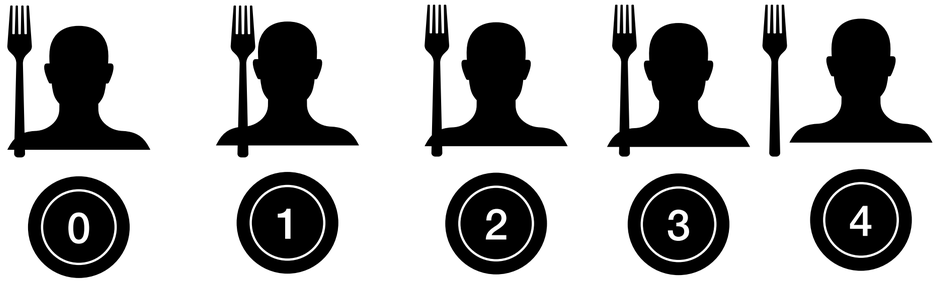
make lock order in t1
and t2 the same

```
void *t2(void *arg){
   pthread_mutex_lock(&m1);
   pthread_mutex_lock(&m2);
   pthread_mutex_unlock(&m2);
   pthread_mutex_unlock(&m1);
    return NULL;
}
```

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
int main(void){
    pthread_t p1, p2;
    pthread_create(&p1, NULL, t1, NULL);
    pthread_create(&p2, NULL, t2, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    return 0;
}
```

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```




```
 void get_forks(int p) {
   sem_wait(&forks[left[p]]);
   sem_wait(&forks[right[p]]);
 }
```

How can we apply ordering here?

The philosophers are always grabbing left and then right fork

```
 void put_forks(int p) {
   sem_post(&forks[left[p]]);
   sem_post(&forks[right[p]]);
 }
```

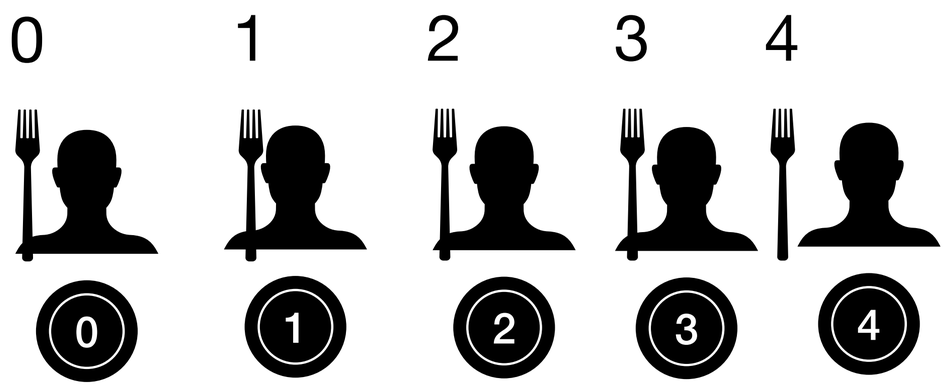
```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
 void get_forks(int p) {
   sem_wait(&forks[left[p]]);
   sem_wait(&forks[right[p]]);
 }
```

```
 void put_forks(int p) {
   sem_post(&forks[left[p]]);
   sem_post(&forks[right[p]]);
 }
```

this is similar to our previous example with threads waiting for each other



let's number the forks first

get_forks(0) -> P0 grabs fork 0 then fork 1
get_forks(1) -> P1 grabs fork 1 then fork 2
get_forks(2) -> P2 grabs fork 2 then fork 3
get_forks(3) -> P3 grabs fork 3 then fork 4
get_forks(4) -> P4 grabs fork 4 then fork 0
what's different about P4?

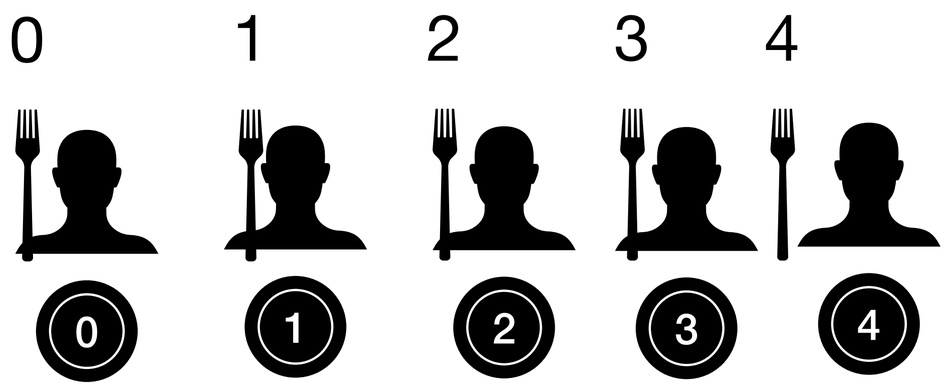
P0-P3 grab locks in increasing order
P4 is the only one decreasing

```
while (1) {
  think(); // no forks needed
  get_forks(p); // get both forks
  eat();
  put_forks(p); // put forks down
```

```
int left(int p) {return p; }
int right(int p) {return (p+1) % 5;}
```

```
void get_forks(int p) {
  if (p == 4) {
    sem_wait(&forks[right[p]]);
    sem_wait(&forks[left[p]]);
  } else {
    sem_wait(&forks[left[p]]);
    sem_wait(&forks[right[[p]]);
```

```
 void put_forks(int p) {
   sem_post(&forks[left[p]]);
   sem_post(&forks[right[p]]);
 }
```



We can fix this by enforcing the same lock order

need to add a condition for P4

get_forks(0) -> P0 grabs fork 0 then fork 1
get_forks(1) -> P1 grabs fork 1 then fork 2
get_forks(2) -> P2 grabs fork 2 then fork 3
get_forks(3) -> P3 grabs fork 3 then fork 4
get_forks(4) -> P4 grabs fork **0** then fork **4**

Even if the other 4 will grab their forks at the same time, the 5th one will wait for a different fork -> no deadlock!

Prevention: No preemption

- Can acquire locks **temporarily**
  - `pthread_mutex_trylock`
    - if lock is available - grabs the lock and returns 0
    - if lock is taken - returns an error
      - will need to backtrack to a different location in the code

Example from Lecture #15

```
void *t1(void *arg) {
    while (1) {
        pthread_mutex_lock(&m1);
        usleep(100000);
        if (pthread_mutex_trylock(&m2) == 0) {
            pthread_mutex_unlock(&m2);
            pthread_mutex_unlock(&m1);
            break;
        }
       pthread_mutex_unlock(&m1);
       usleep(1000);
    }
    return NULL;
}
```

How can we use mutex_trylock here?

t1 takes m1 and tries m2

if m2 is not available, releases m1 and tries again

neither of them will hold the lock at the same time while waiting for another

```
void *t2(void *arg) {
    while (1) {
        pthread_mutex_lock(&m2);
        if (pthread_mutex_trylock(&m1) == 0) {
            pthread_mutex_unlock(&m1);
            pthread_mutex_unlock(&m2);
            break;
        }
        pthread_mutex_unlock(&m2);
        usleep(1000);
    }
    return NULL;
}
```

t1 takes m2 and tries m1

if m1 is not available, releases m2 and tries again

don't need to change the order of locks

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
int main(void){
    pthread_t p1, p2;
    pthread_create(&p1, NULL, t1, NULL);
    pthread_create(&p2, NULL, t2, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    return 0;
}
```

Prevention: No preemption

- Can acquire locks **temporarily**
  - pthread_mutes_trylock
    - if lock is available - grabs the lock and returns 0
    - if lock is taken - returns an error
- Technically, this is **self-preemption**
  - there is **no external actor** taking the lock away
  - the thread **gives it up**
- Cons: **difficult to implement** in more complex cases
  - often hard to determine what to execute when the lock acquisition fails

Prevention: Lock-and-wait

- Can acquire all locks atomically
  - aka put another lock around multiple lock acquisition code

Example from Lecture #15

How can we apply hold-and-wait prevention?

add one more lock

now order of acquisition does not matter

```
void *t1(void *arg) {
    pthread_mutex_lock(prevention);
    pthread_mutex_lock(&m1);
    usleep(100000);
    pthread_mutex_lock(&m2);
pthread_mutex_lock(prevention);
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}
```

```
void *t2(void *arg) {

  pthread_mutex_lock(prevention);
  pthread_mutex_lock(&m1);
  pthread_mutex_lock(&m2);
  pthread_mutex_lock(prevention);

  pthread_mutex_unlock(&m2);
  pthread_mutex_unlock(&m1);
  return NULL;
}
```

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t prevention = PTHREAD_MUTEX_INITIALIZER
int main(void){
    pthread_t p1, p2;

    pthread_create(&p1, NULL, t1, NULL);
    pthread_create(&p2, NULL, t2, NULL);

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);

    return 0;
}
```

Prevention: Lock-and-wait

- Can acquire all locks atomically
  - aka put another lock around multiple lock acquisition code
- Pros: no deadlocks guaranteed
- Cons: no concurrency either

Prevention: Mutual exclusion

- can use lock-free and wait-free approaches
  - e.g. hardware atomic instructions

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

each thread creates a new node

this is just to check if the malloc failed

initializes the node

gets the lock

makes this node first and current head node its next

releases the lock

concurrent list insertion with locks

Prevention: Mutual exclusion

- can use lock-free and wait-free approaches
  - e.g. hardware atomic instructions

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

concurrent list insertion with locks

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0)
}
```

concurrent list insertion with atomic operation

concurrent list insertion with locks

A

value = 15

a	
?	next

A creates a node

head	
3	next

A

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

A

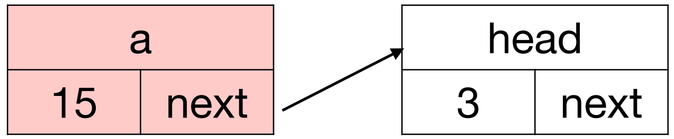
value = 15

A creates a node

makes it point to the head

A

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```



A

value = 15

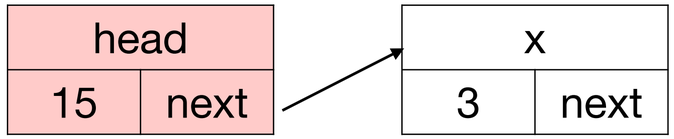
A creates a node

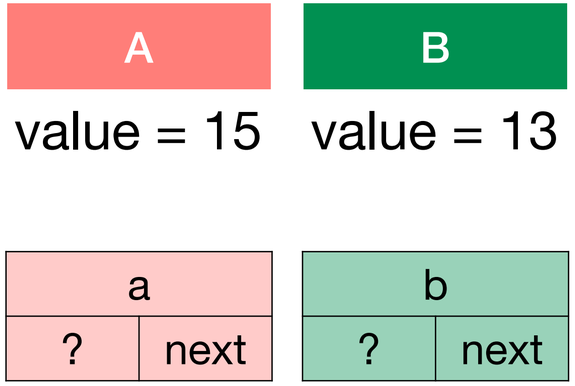
makes it point to the head

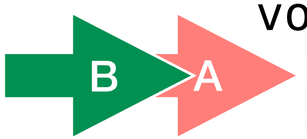
makes the new node head

A

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

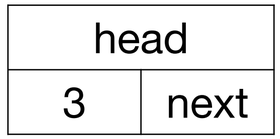


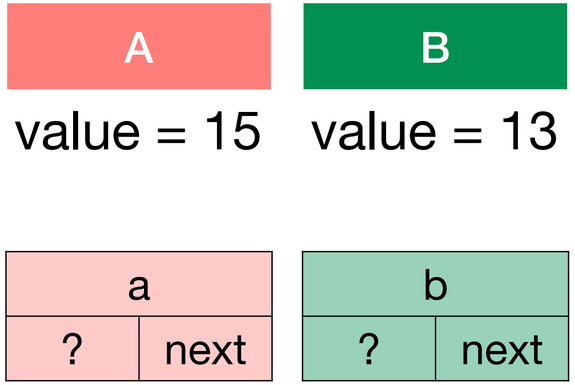


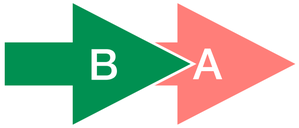



```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

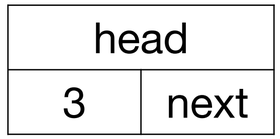
A and B create their nodes





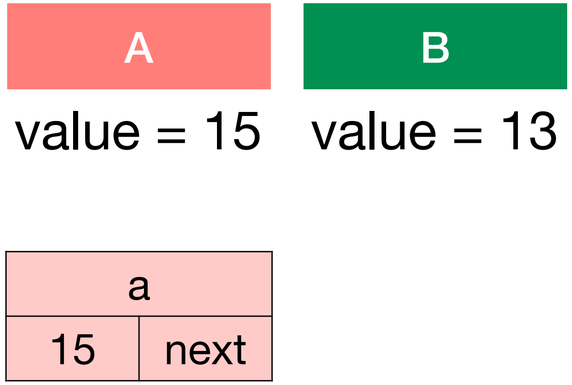



```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```



A and B create their nodes

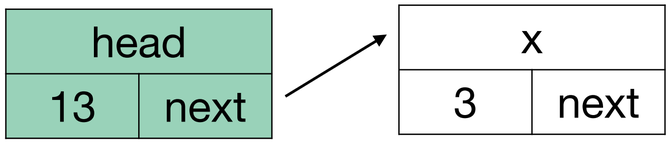
try to get a lock




```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

A

B



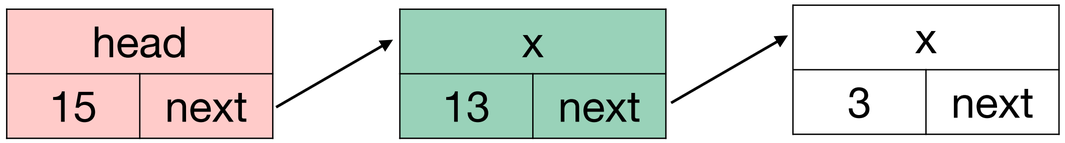
A and B create their nodes

try to get a lock

B inserts its node first

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  pthread_mutex_lock(listlock);
  n->next = head;
  head = n;
  pthread_mutex_unlock(listlock);
}
```

A



A and B create their nodes

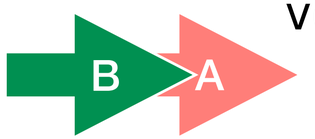
try to get a lock

B inserts its node first

then A inserts its

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0




```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A and B create their nodes

A

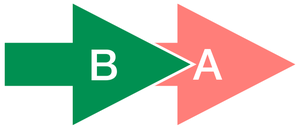
value = 15

B

value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0




```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A and B create their nodes

n	
13	next

n	
15	next

head	
3	next

B

value = 15

A

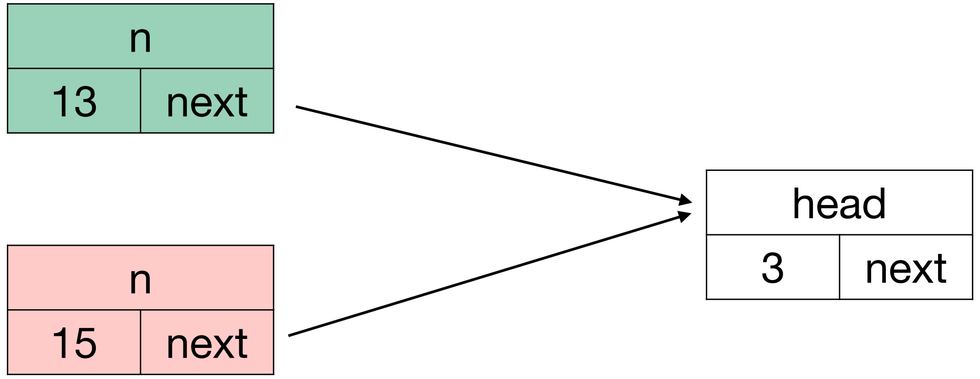
value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

B A



A and B create their nodes

assign next pointer to cur head

A

value = 15

B

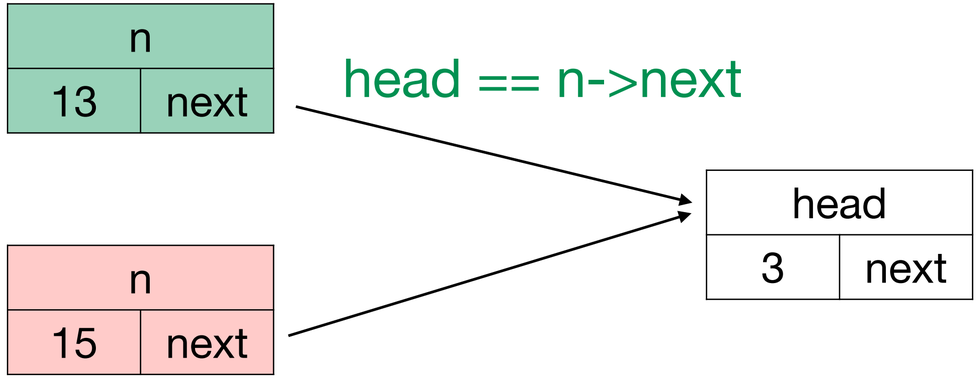
value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

B A



A and B create their nodes

assign next pointer to cur head and evaluate CAS

A

value = 15

B

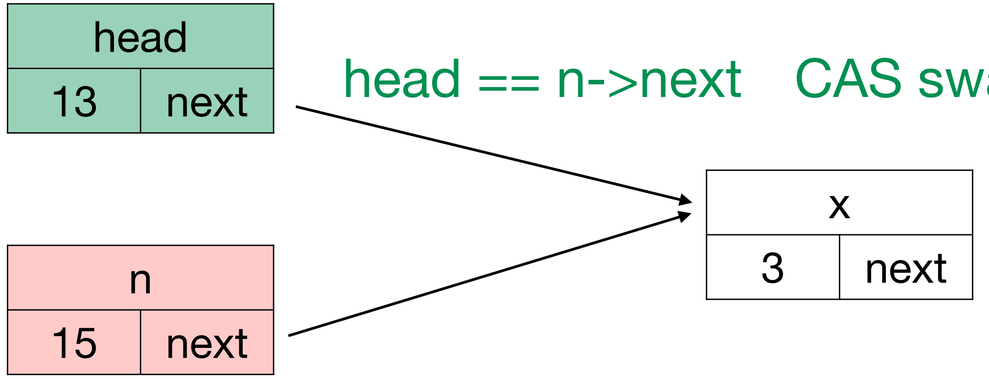
value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

B A



head == n->next CAS swaps and returns 1

A and B create their nodes

assign next pointer to cur head
and evaluate CAS
B inserts first and leaves

A

value = 15

B

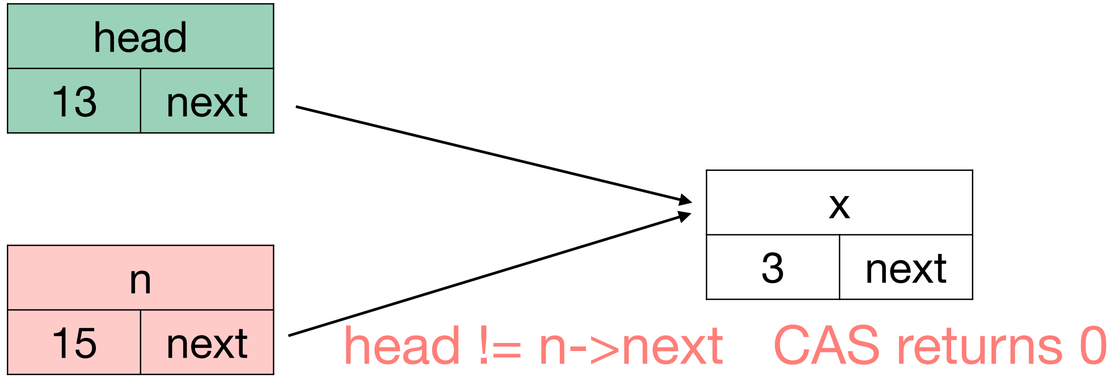
value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A



A and B create their nodes

assign next pointer to cur head
and evaluate CAS
B inserts first and leaves
A has to do another round

A

value = 15

B

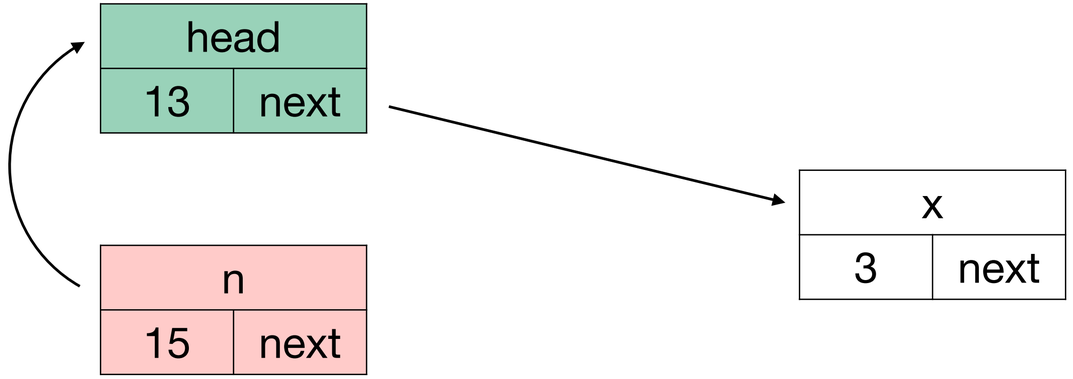
value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A



A and B create their nodes

assign next pointer to cur head
and evaluate CAS
B inserts first and leaves
A has to do another round

A

value = 15

B

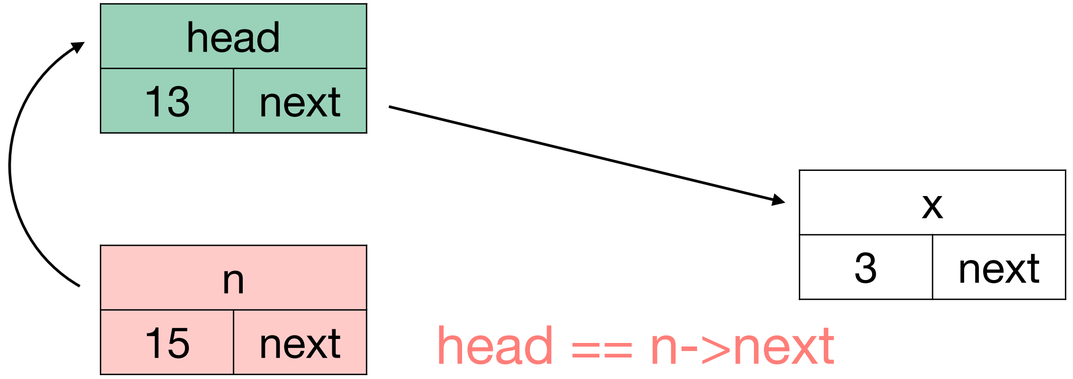
value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A



head == n->next

A and B create their nodes

assign next pointer to cur head
and evaluate CAS
B inserts first and leaves
A has to do another round
A points to the cur head

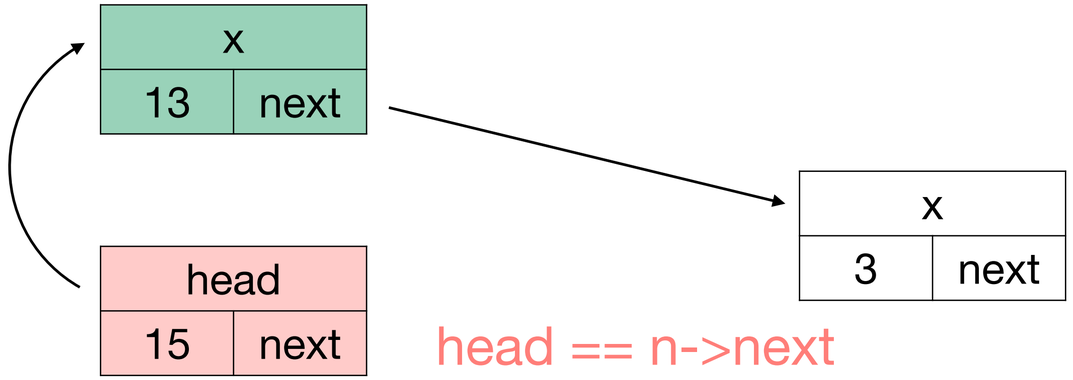
T1	T2
value = 15	value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

T1



head == n->next

CAS returns 1 and swaps values

A and B create their nodes

assign next pointer to cur head
and evaluate CAS
B inserts first and leaves
A has to do another round
A points to the cur head
and inserts

A

value = 15

B

value = 13

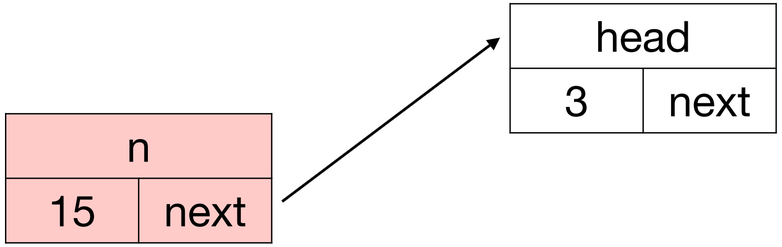
```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps
old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A

this can still break



A

value = 15

B

value = 13

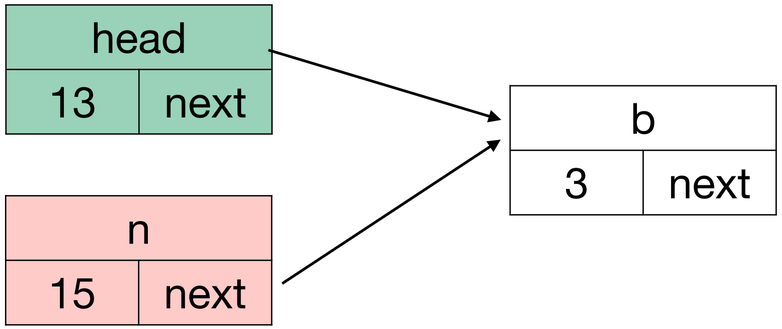
```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A

B



this can still break

if B swoops in between A updating its pointer and running CAS operation

if this happens often, A will starve

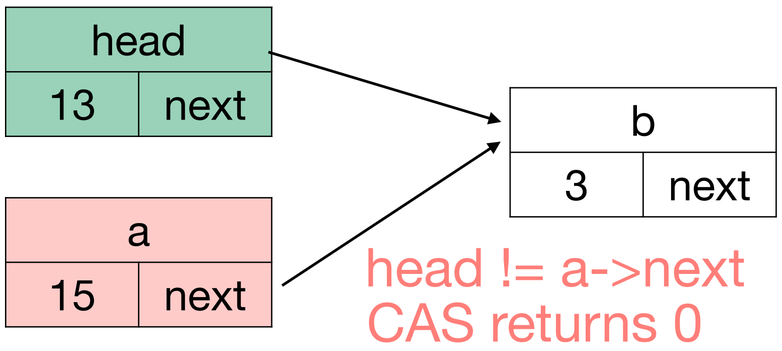
T1	T2
value = 15	value = 13

```
CompareAndSwap(*ptr, old, new)
if *ptr  == old
  *ptr = new; return 1
return 0
```

compares val at ptr to old
if they are the same, swaps old with new, returns 1
otherwise, no swap, returns 0

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (CompareAndSwap(&head, n->next, n) == 0);
}
```

A



this can still break

if B swoops in between A updating its pointer and running CAS operation

if this happens often, A will starve

Prevention

- Deadlock will happen if all **4 deadlock conditions** are satisfied
- Breaking **any** of them will prevent deadlocks
- **Circular wait** - total or partial ordering of lock acquisition
  - cons: hard to implement in complex codebases, developers need to follow the same rules
- **Hold-and-wait** - treat lock acquisition itself as critical section and lock it up
  - cons: reduces concurrency, hard to implement in complex codebases
- **No preemption** - hold the lock temporarily
  - cons: can result in a livelock, retry can be difficult to implement in complex codebases
- **Mutual exclusion** - avoid locks altogether and use atomic hardware ops
  - cons: starvation still possible

Avoidance

- Anticipate where the deadlocks may occur and schedule around them

Avoidance

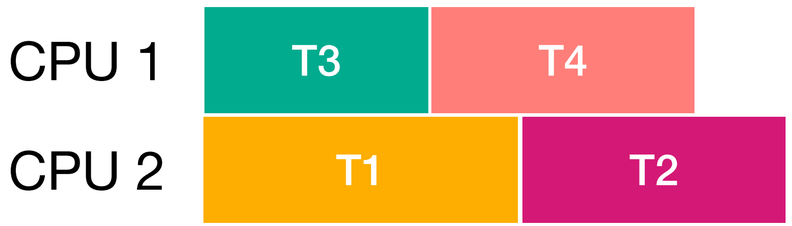
- Anticipate where the deadlocks may occur and schedule around them

4 processes are using 2 locks

	T1	T2	T3	T4
L1	yes	yes	no	no
L2	yes	yes	yes	no

T1 and T2 should not run concurrently

T4 and T3 are safe to run concurrently with any other process



Avoidance

- Anticipate where the deadlocks may occur and schedule around them

4 processes are using 2 locks

	T1	T2	T3	T4
L1	yes	yes	yes	no
L2	yes	yes	yes	no

How about now?

T1, T2, and T3 should not run concurrently

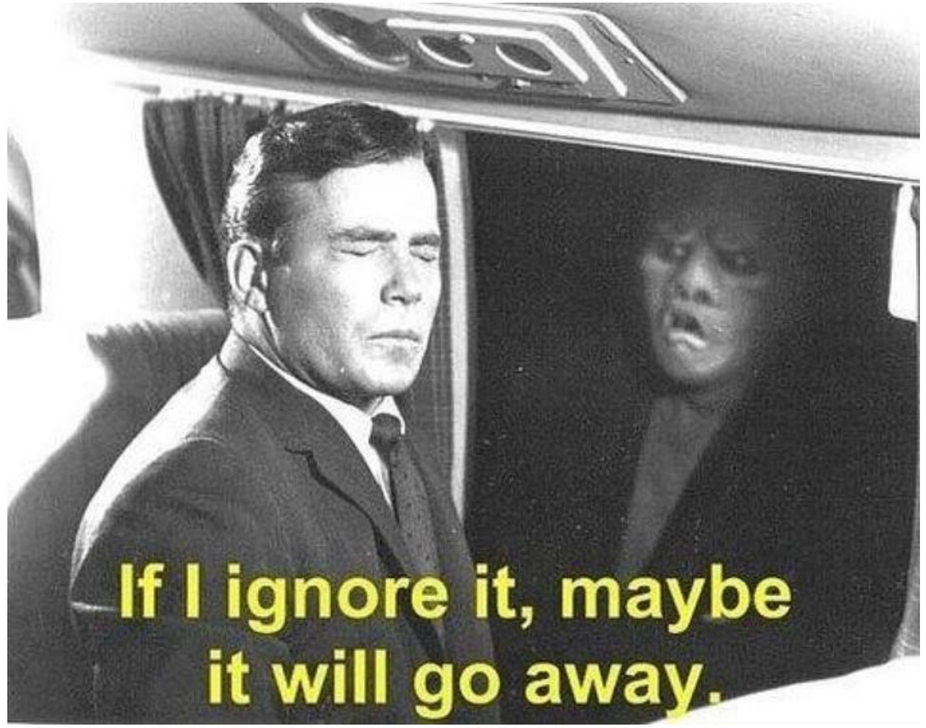
no deadlocks, but also no concurrency

Avoidance

- **Anticipate** where the deadlocks may occur and schedule around them
- Pros: avoids deadlocks without changing the code
- Cons:
  - can reduce concurrency and performance
  - impractical
    - requires information about all tasks and locks

Recovery

- If deadlocks are rare, let them happen
  - aka the ostrich algorithm
- Detect and recover
  - Identify this is indeed a deadlock
    - build a resource allocation graph
    - if there are cycles in it -> found a deadlock
  - Kill the process
  - Reboot the machine



Summary

- Concurrency can cause issues that are often hard to find and fix
  - race conditions
  - deadlocks
  - livelocks

Summary

- Dealing with deadlocks
  - Prevention
    - Do not allow conditions for the deadlocks to arise
    - Requires rewriting the code
  - Avoidance
    - Use scheduling to schedule threads such that deadlock cannot occur
    - Don’t need to rewrite the code
  - Recovery
    - Allow deadlocks to happen, detect them, and recover

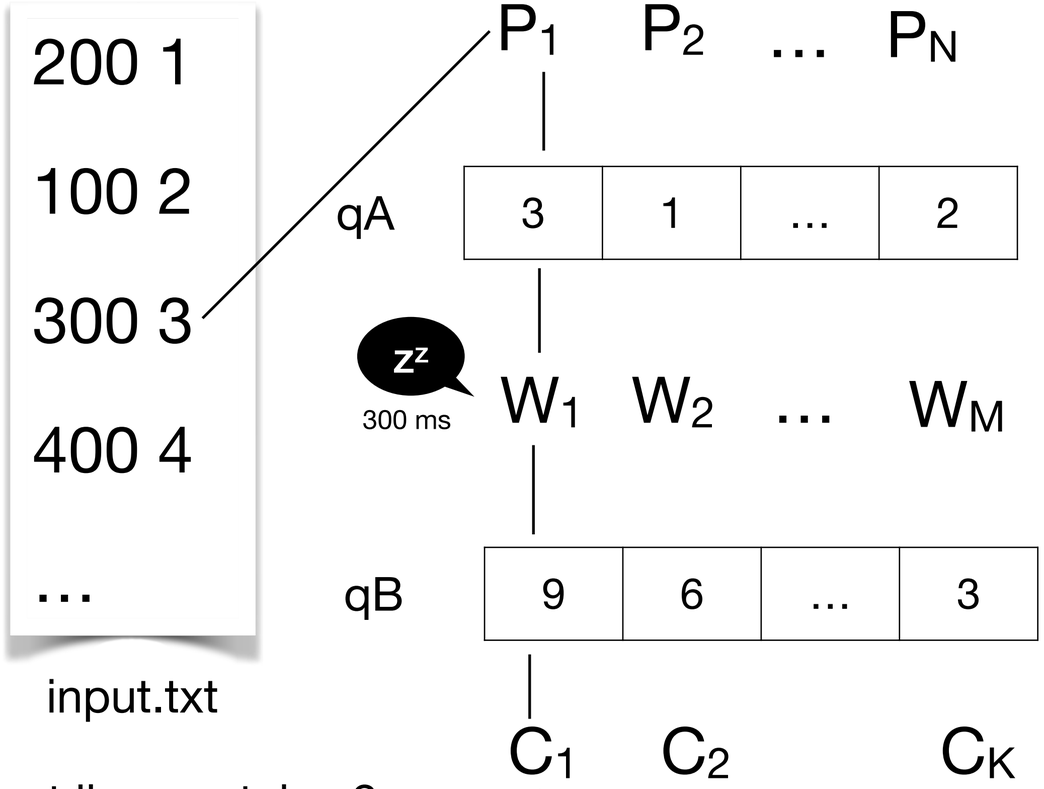
Reading

- Chapter 32 Common concurrency problems
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-bugs.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/threads-bugs

A4

- Due **March 30th @ 11:59 PM EST**
- A variation of the consumer-producer problem
  - producers - read values from the file - delay, value
  - workers - sleep and multiply value by 3
  - consumers - print the result in the original order

A4



each input line contains 2 integers: sleep and value

N producer threads read input line by line concurrently

and enqueue values in queue A

M worker threads dequeue values from queue A

sleep for the prescribed amount

multiply the values by 3 and enqueue them in queue B

K producer threads dequeue the values from queue B and print them **in same order as they were in the input file**

```
>>> ./pipeline input.txt N M K
3
6
9
...
```

A4

- The assignment is trivial to solve sequentially with one thread
- Your goal is to maximize concurrency while maintaining correctness
- Not allowed:
  - **sequential solutions**, e.g. blocking all threads except one that does the work
  - must use **2 concurrent queues**
  - **checking input size** in advance to set queue sizes
    - everything must be allocated **dynamically**
    - producers should **signal** to other threads when the input is finished
  - **buffering** all outputs and **sorting** them at the end
    - consumers must print **as soon as they can**
      - i.e. the item is processed and previous items are printed already
  - use of any synchronization functions **not** introduced in class
    - only pthread mutexes, cond variables and/or semaphores
    - no monitors, barriers, external libraries, etc.

all code will be manually checked

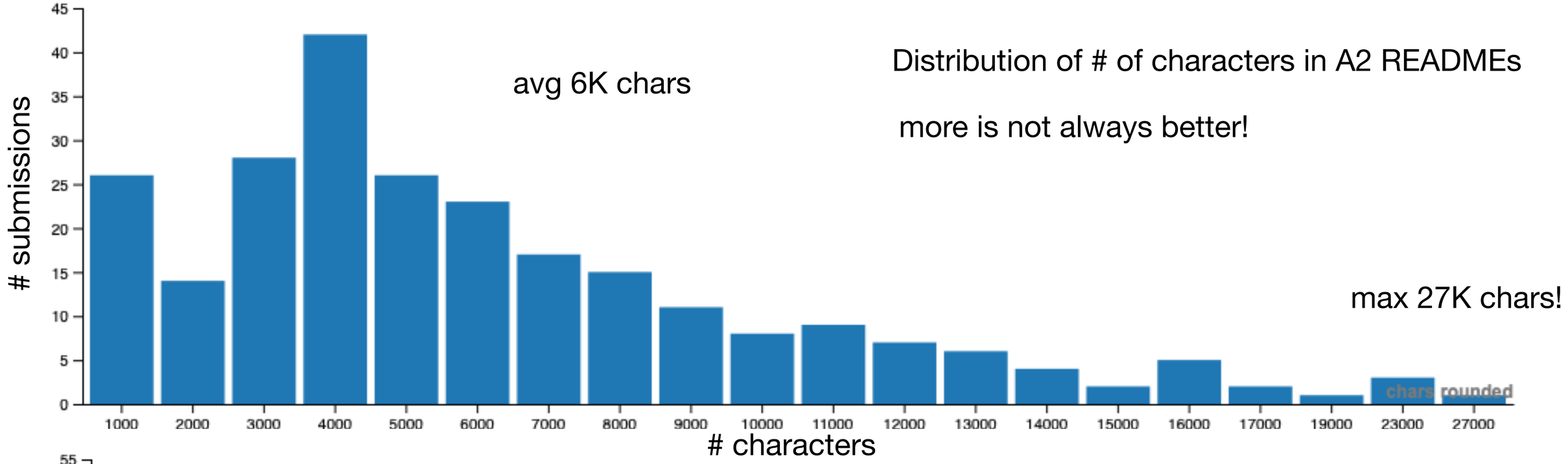
not following **any or all** of these conditions will result in 0 grade regardless of the autograder output

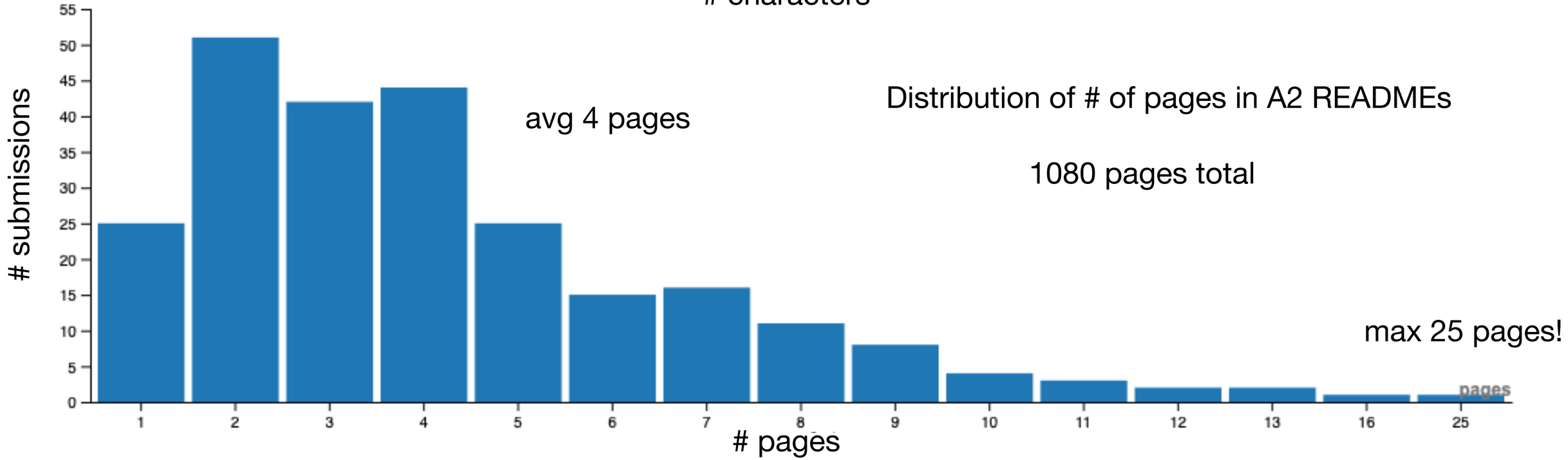
A4

- Autograder takes about 10 minutes to generate a report
  - sequential inputs (e.g. 1 2 3) with varying delay patterns
  - large inputs to reveal concurrency issues/deadlocks
  - different patterns of P/W/C thread numbers
- Trivial to generate test cases
  - you can do it yourself before submitting the solution

A4 README

- 10 marks for explaining the implementation
- 10 marks for installation and usage
- Character limit
  - 2,000 characters for the implementation
  - 4,000 characters total


Distribution of # of characters in A2 READMEs
avg 6K chars
more is not always better!
max 27K chars!
submissions
characters
chars rounded
0
5
10
15
20
25
30
35
40
45
1000
2000
3000
4000
5000
6000
7000
8000
9000
10000
11000
12000
13000
14000
15000
16000
17000
19000
23000
27000


Distribution of # of pages in A2 READMEs
avg 4 pages
1080 pages total
max 25 pages!
submissions
pages
pages
0
5
10
15
20
25
30
35
40
45
50
55
1
2
3
4
5
6
7
8
9
10
11
12
13
16
25

A4

- Start early
- Give yourself enough time to study for the final

Quiz 16

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999